

4. ПРОЕКТИРОВАНИЕ ИНТЕРФЕЙСОВ (DESIGNING INTERFACES)

У каждого проекта, каким бы большим или маленьким он ни был, есть API. На самом деле обычно их несколько. Некоторые из них обращены к пользователю — например, HTTP-эндпоинт или интерфейс командной строки, а некоторые обращены к разработчику — например, публичный интерфейс библиотеки. Помимо этого, у крейтов (crate) Rust есть ещё и ряд внутренних интерфейсов: каждый тип, трейт (trait) и граница модуля имеют свой собственный миниатюрный API, с которым взаимодействует остальной ваш код. По мере того как ваша кодовая база растёт в размере и сложности, вы обнаружите, что стоит вложить некоторые размышления и заботу в то, как вы проектируете даже внутренние API, чтобы опыт использования и сопровождения кода со временем оставался настолько приятным, насколько это возможно.

В этой главе мы рассмотрим некоторые из наиболее важных соображений при написании идиоматичных интерфейсов на Rust — независимо от того, являются ли пользователями этих интерфейсов ваш собственный код или другие разработчики, использующие вашу библиотеку. По сути, всё сводится к четырём принципам: ваши интерфейсы должны быть *без сюрпризов* (*unsurprising*), *гибкими* (*flexible*), *очевидными* (*obvious*) и *ограниченными* (*constrained*). Я разберу каждый из этих принципов по очереди, чтобы дать некоторые рекомендации по написанию надёжных и удобных интерфейсов.

Я настоятельно рекомендую ознакомиться с Rust API Guidelines (<https://rust-lang.github.io/api-guidelines/>) после прочтения этой главы. Там есть отличный чек-лист, которому можно следовать, с подробным разбором каждой рекомендации. Многие из рекомендаций в этой главе также проверяются инструментом `cargo clippy`, который вам стоит начать запускать на своём коде, если вы ещё этого не делаете. Я также призываю вас прочитать Rust RFC 1105 (<https://rust-lang.github.io/rfcs/1105-api-evolution.html>) и главу *The Cargo Book* о совместимости по SemVer (<https://doc.rust-lang.org/cargo/reference/semver.html>), которые охватывают то, что является и что не является ломающим изменением в Rust.

Без сюрпризов (Unsurprising)

Принцип наименьшего удивления, иначе известный как закон наименьшего изумления, часто упоминается в программной инженерии, и он справедлив и для интерфейсов Rust. Где это возможно, ваши интерфейсы должны быть достаточно интуитивными, чтобы, когда пользователю приходится угадывать, он обычно угадывал правильно. Конечно, не всё в вашем приложении будет сразу интуитивно понятным таким образом, но всё, что *может* быть без сюрпризов, таким и должно быть. Основная идея здесь — держаться близко к тем вещам, которые пользователь, скорее всего, уже знает, чтобы ему не приходилось заново осваивать концепции иным способом, чем тот, к которому он привык. Так вы сэкономите его умственные силы для того, чтобы разобраться в вещах, которые действительно специфичны для вашего интерфейса.

Существует множество способов сделать ваши интерфейсы предсказуемыми. Здесь мы рассмотрим, как использовать именование, общие трейты и эргономичные трюки с трейтами, чтобы помочь пользователю.

Практики именования (Naming Practices)

Пользователь вашего интерфейса столкнётся с ним сначала через его имена; он сразу же начнёт делать выводы о вещах из имён типов, методов, переменных, полей и библиотек, которые встретит. Если ваш интерфейс переиспользует имена для вещей — скажем, методов и типов — из других (возможно, общеизвестных) интерфейсов, пользователь сможет делать определённые предположения о ваших методах и типах. Метод под названием `iter` вероятно принимает `&self` и, вероятно, отдаёт итератор. Метод под названием `into_inner` вероятно принимает `self` и, скорее всего, возвращает некий обёрнутый тип. Тип под названием `SomethingError` вероятно реализует `std::error::Error` и появляется в различных `Result`. Переиспользуя общие имена для одной и той же цели, вы облегчаете пользователю задачу угадывания того, что делают те или иные вещи, и позволяете ему легче понять то, что в вашем интерфейсе отличается.

Следствием из этого является то, что вещи, разделяющие имя, *должны* на самом деле работать одинаково. Иначе — например, если ваш метод `iter` принимает `self`, или если ваш тип `SomethingError` не реализует `Error` — пользователь, скорее всего, напишет некорректный код, исходя из того, как он

ожидает, что интерфейс будет работать. Он будет удивлён и расстроен и будет вынужден потратить время на выяснение того, чем ваш интерфейс отличается от его ожиданий. Когда мы можем избавить пользователя от такого рода тренировок, нам следует это делать.

Общие трейты для типов (Common Traits for Types)

Пользователи в Rust также будут исходить из главного предположения, что всё в интерфейсе «просто работает». Они ожидают возможности напечатать любой тип с помощью `{:?}` и отправить что угодно в другой поток, и они ожидают, что каждый тип будет `Clone`. Где это возможно, нам снова следует избегать удивления пользователя и охотно реализовывать большинство стандартных трейтов, даже если они нам не нужны прямо сейчас.

Из-за правил когерентности (coherence), обсуждавшихся в главе 3, компилятор не позволит пользователям реализовывать эти трейты, когда они им понадобятся. Пользователям не разрешается реализовывать чужой трейт (например, `Clone`) для чужого типа, такого как один из вашего интерфейса. Вместо этого им пришлось бы обернуть тип вашего интерфейса в свой собственный тип, и даже тогда написать разумную реализацию без доступа к внутренностям типа может быть довольно сложно.

Первым среди этих стандартных трейтов идёт трейт `Debug`. Почти каждый тип может и должен реализовывать `Debug`, даже если он лишь печатает имя типа. Использование `#[derive(Debug)]` часто является лучшим способом реализовать трейт `Debug` в вашем интерфейсе, но имейте в виду, что все выводимые (derived) трейты добавляют одну и ту же границу для любых обобщённых параметров. Вы также можете просто написать собственную реализацию, задействовав различные вспомогательные методы `debug_` на `fmt::Formatter`.

В тесном втором эшелоне идут авто-трейты (auto-traits) Rust `Send` и `Sync` (и, в меньшей степени, `Unpin`). Если тип не реализует один из этих трейтов, на то должна быть очень веская причина. Тип, который не `Send`, нельзя поместить в `Mutex` и нельзя использовать даже транзитивно в приложении, содержащем пул потоков. Тип, который не `Sync`, нельзя разделять через `Arc` или помещать в статическую переменную. Пользователи привыкли ожидать, что типы *просто работают* в этих контекстах, особенно в асинхронном мире, где почти всё выполняется на пуле потоков, и они будут раздосадованы, если вы не обеспечите реализацию этих трейтов вашими типами. Если ваши типы не

могут реализовать их, чётко обозначьте этот факт и причину — это хорошо задокументируйте!

Следующий набор почти универсальных трейтов, которые вам стоит реализовать — это `Clone` и `Default`. Эти трейты можно легко вывести или реализовать, и для большинства типов их реализация имеет смысл. Если ваш тип не может реализовать эти трейты, обязательно укажите это в документации, поскольку пользователи обычно ожидают, что смогут легко создавать новые экземпляры типов по своему усмотрению. Если они не могут, это их удивит.

Ещё на одну ступень ниже в иерархии ожидаемых трейтов идут трейты сравнения: `PartialEq`, `PartialOrd`, `Hash`, `Eq` и `Ord`. Трейт `PartialEq` особенно желателен, потому что пользователи в какой-то момент неизбежно получают два экземпляра вашего типа, которые захотят сравнить с помощью `==` или `assert_eq!`. Даже если ваш тип будет сравниваться на равенство только с тем же самым экземпляром типа, стоит реализовать `PartialEq`, чтобы позволить вашим пользователям использовать `assert_eq!`.

`PartialOrd` и `Hash` более специализированы и могут применяться не так широко, но где это возможно, вы тоже захотите их реализовать. Это особенно верно для типов, которые пользователь может захотеть использовать как ключ в отображении (`map`), или для типа, который он может захотеть дедуплицировать с помощью любого из множественных типов-множеств `std::collection`, поскольку они, как правило, требуют этих границ. `Eq` и `Ord` несут дополнительные семантические требования к операциям сравнения реализующего типа сверх тех, что у `PartialEq` и `PartialOrd`. Они хорошо задокументированы в документации этих трейтов, и вам следует реализовывать их *только* в том случае, если вы уверены, что эта семантика действительно применима к вашему типу.

Наконец, для большинства типов имеет смысл реализовать трейты `Serialize` и `Deserialize` крейта `serde`. Их можно легко вывести, и крейт `serde_derive` даже содержит механизмы для переопределения сериализации лишь для одного поля или варианта перечисления. Поскольку `serde` — это сторонний крейт, вы можете не захотеть добавлять обязательную зависимость от него. Поэтому большинство библиотек предпочитают предоставлять фичу (feature) `serde`, которая добавляет поддержку `serde` только тогда, когда пользователь сам её включает.

Вы можете задаться вопросом, почему я не включил выводимый трейт `Copu` в этот раздел. Есть две вещи, которые отличают `Copu` от других упомянутых трейтов. Первая — это то, что пользователи обычно не ожидают, что типы будут `Copu`; совсем наоборот, они склонны ожидать, что если им нужны две копии чего-либо, они должны вызвать `clone`. `Copu` меняет семантику перемещения значения данного типа, что может удивить пользователя. Это связано со вторым наблюдением: типу очень легко *перестать* быть `Copu`, потому что типы `Copu` сильно ограничены. Тип, который начинался простым, легко может в итоге начать хранить `String` или какой-то другой не-`Copu` тип. Если это произойдёт, и вам придётся удалить реализацию `Copu`, это будет обратно несовместимым изменением. Гораздо реже вам придётся удалять реализацию `Clone`, так что это менее обременительное обещание.

Эргономичные реализации трейтов (Ergonomic Trait Implementations)

Rust не реализует автоматически трейты для ссылок на типы, которые реализуют трейты. Иначе говоря, вы, как правило, не можете вызвать `fn foo<T: Trait>(t: T)` с `&Bar`, даже если `Bar: Trait`. Это потому, что `Trait` может содержать методы, которые принимают `&mut self` или `self`, которые, очевидно, нельзя вызвать на `&Bar`. Тем не менее, такое поведение может очень удивить пользователя, который видит, что у `Trait` есть только методы с `&self`!

По этой причине, когда вы определяете новый трейт, вы обычно захотите предоставить бланкетные (blanket) реализации, где это уместно, для этого трейта для `&T where T: Trait`, `&mut T where T: Trait` и `Box<T> where T: Trait`. Возможно, вы сможете реализовать лишь некоторые из них, в зависимости от того, какие получатели (receivers) имеют методы `Trait`. Многие из трейтов в стандартной библиотеке имеют похожие реализации, именно потому что это ведёт к меньшему числу сюрпризов для пользователя.

Итераторы — это ещё один случай, когда вы часто захотите специально добавить реализации трейтов для ссылок на типы. Для любого типа, по которому можно итерироваться, рассмотрите возможность реализации `IntoIterator` как для `&MyType`, так и для `&mut MyType`, где это применимо. Это позволяет циклам `for` работать с заимствованными экземплярами вашего типа «из коробки», именно так, как пользователи бы и ожидали.

Типы-обёртки (Wrapper Types)

В Rust нет наследования объектов в классическом смысле. Однако трейт `Deref` и его родственник `AsRef` оба предоставляют нечто немного похожее на наследование. Эти трейты позволяют вам иметь значение типа `T` и вызывать методы некоторого типа `U`, обращаясь к ним напрямую на значении типа `T`, если `T: Deref<Target = U>`. Для пользователя это ощущается как магия и в целом замечательно.

Если вы предоставляете относительно прозрачный тип-обёртку (например, `Arc`), есть большая вероятность, что вы захотите реализовать `Deref`, чтобы пользователи могли вызывать методы внутреннего типа, просто используя оператор `.`. Если доступ к внутреннему типу не требует какой-либо сложной или потенциально медленной логики, вам также стоит рассмотреть реализацию `AsRef`, которая позволяет пользователям легко использовать `&WrapperType` как `&InnerType`. Для большинства типов-обёрток вы также захотите реализовать `From<InnerType>` и `Into<InnerType>`, где это возможно, чтобы ваши пользователи могли легко добавлять или удалять вашу обёртку.

Возможно, вы также встречали трейт `Borrow`, который ощущается очень похожим на `Deref` и `AsRef`, но на самом деле представляет собой немного другого зверя. В частности, `Borrow` заточен под гораздо более узкий сценарий использования: позволить вызывающему предоставить любой из множества по сути идентичных вариантов одного и того же типа. Его, пожалуй, можно было бы назвать `Equivalent`. Например, для `HashSet<String>` `Borrow` позволяет вызывающему предоставить либо `&str`, либо `&String`. Хотя того же можно было бы достичь с помощью `AsRef`, это было бы небезопасно без дополнительного требования `Borrow`, что целевой тип реализует `Hash`, `Eq` и `Ord` точно так же, как реализующий тип. У `Borrow` также есть бланкетная реализация `Borrow<T>` для `T`, `&T` и `&mut T`, что делает удобным его использование в границах трейтов для приёма либо владеемых, *либо* заимствованных значений данного типа. В общем, `Borrow` предназначен только для случая, когда ваш тип по сути эквивалентен другому типу, тогда как `Deref` и `AsRef` предназначены для более широкой реализации для всего, в роли чего ваш тип может «выступить».

DEREF И СОБСТВЕННЫЕ МЕТОДЫ (DEREF AND INHERENT METHODS) Магия вокруг оператора точки и `Deref` может становиться запутанной и удивительной, когда у `T` есть методы, которые принимают `self`. Например, для значения `t: T` неясно, фробнизирует ли

```
t.frobnicate() сам T или нижележащий U!
```

По этой причине типам, которые позволяют прозрачно вызывать методы на некотором внутреннем типе, не известном заранее, следует избегать собственных (inherent) методов. Для Vec нормально иметь метод push, даже несмотря на то что он разыменовывается в срез (slice), поскольку вы знаете, что у срезов в ближайшее время не появится метод push. Но если ваш тип разыменовывается в управляемый пользователем тип, любой собственный метод, который вы добавите, может уже существовать на этом управляемом пользователем типе и тем самым вызвать проблемы. В таких случаях отдавайте предпочтение статическим методам вида `fn frobnicate(t: T)`. Тогда `t.frobnicate()` всегда вызывает `U::frobnicate`, а `T::frobnicate(t)` можно использовать для фробнициирования самого T.

Гибкий (Flexible)

Каждый фрагмент кода, который вы пишете, неявно или явно содержит контракт. Контракт состоит из набора требований и набора обещаний. Требования — это ограничения на то, как код можно использовать, а обещания — это гарантии того, как код можно использовать. При проектировании нового интерфейса стоит тщательно подумать об этом контракте. Хорошее правило — избегать наложения ненужных ограничений и давать только те обещания, которые вы можете сдержать. Добавление ограничений или удаление обещаний обычно требует изменения мажорной семантической версии и, скорее всего, сломает код в другом месте. Ослабление ограничений или предоставление дополнительных обещаний, с другой стороны, обычно обратно совместимо.

В Rust ограничения обычно принимают форму границ трейтов и типов аргументов, а обещания принимают форму реализаций трейтов и возвращаемых типов. Например, сравните три сигнатуры функций в листинге 4-1.

```
fn frobnicate1(s: String) -> String
fn frobnicate2(s: &str) -> Cow<'_, str>
fn frobnicate3(s: impl AsRef<str>) -> impl AsRef<str>
```

Листинг 4-1: Похожие сигнатуры функций с разными контрактами

Все эти три сигнатуры функций принимают строку и возвращают строку, но делают это в рамках очень разных контрактов.

Первая функция требует, чтобы вызывающий владел строкой в форме типа `String`, и обещает, что вернёт владеемую `String`. Поскольку контракт требует, чтобы вызывающий выделил память, и требует от нас вернуть владеемую `String`, мы не можем позднее сделать эту функцию свободной от выделений памяти обратно совместимым образом.

Вторая функция ослабляет контракт: вызывающий может предоставить любую ссылку на строку, так что пользователю больше не нужно выделять память или отказываться от владения `String`. Она также обещает вернуть `std::borrow::Cow`, имея в виду, что она может вернуть либо ссылку на строку, либо владеемую `String`, в зависимости от того, нужно ли ей владеть строкой. Обещание здесь в том, что функция всегда будет возвращать `Cow`, что означает, что мы не можем, скажем, изменить её, чтобы использовать какое-то более оптимизированное представление строки позднее. Вызывающий должен также конкретно предоставить `&str`, так что если у него, скажем, есть уже существующая `String`, ему придётся разыменовать её в `&str`, чтобы вызвать нашу функцию.

Третья функция снимает эти ограничения. Она требует лишь, чтобы пользователь передал тип, который может произвести ссылку на строку, и обещает лишь, что возвращаемое значение может произвести ссылку на строку.

Ни одна из этих сигнатур функций не *лучше* других. Если вам нужно владение строкой в функции, вы можете использовать первый тип аргумента, чтобы избежать лишнего копирования строки. Если вы хотите позволить вызывающему воспользоваться случаем, когда владеемая строка была выделена и возвращена, вторая функция с возвращаемым типом `Cow` может быть хорошим выбором. Вместо этого я хочу, чтобы вы вынесли из этого следующее: вам следует тщательно подумать о том, к какому контракту вас привязывает ваш интерфейс, потому что изменение его постфактум может оказаться разрушительным.

В оставшейся части этого раздела я приведу примеры решений по проектированию интерфейса, которые часто возникают, и их следствия для контракта вашего интерфейса.

Обобщённые аргументы (Generic Arguments)

Одно очевидное требование, которое ваш интерфейс должен наложить на пользователей — это то, какие типы они должны предоставить вашему коду. Если ваша функция явно принимает `Foo`, пользователь должен владеть `Foo` и отдать его вам. Обойти это никак нельзя. В большинстве случаев имеет смысл использовать обобщённые типы вместо конкретных, чтобы позволить вызывающему передать любой тип, соответствующий тому, что на самом деле нужно вашей функции, а не только конкретный тип. Изменение `&str` в листинге 4-1 на `impl AsRef<str>` — это пример такого ослабления. Один из способов ослаблять требования таким образом — начать с полностью обобщённого аргумента без границ, а затем просто следовать за ошибками компилятора, чтобы обнаружить, какие границы вам нужно добавить.

Однако, если довести это до крайности, такой подход сделал бы каждый аргумент каждой функции своим собственным обобщённым типом, что было бы и трудно читать, и трудно понимать. Нет жёстких и однозначных правил относительно того, когда именно вам следует или не следует делать тот или иной параметр обобщённым, так что используйте свой здравый смысл. Хорошее правило — делать аргумент обобщённым, если вы можете представить себе другие типы, которые пользователь мог бы разумно и часто захотеть использовать вместо того конкретного типа, с которого вы начали.

Вы можете помнить из главы 3, что обобщённый код дублируется для каждой комбинации типов, когда-либо использованных с обобщённым кодом, через мономорфизацию (monomorphization). Учитывая это, идея делать множество аргументов обобщёнными может вызвать у вас беспокойство по поводу чрезмерного раздувания ваших бинарных файлов. В главе 3 мы также обсуждали, как можно использовать динамическую диспетчеризацию (dynamic dispatch) для смягчения этого при (обычно) пренебрежимо малой стоимости производительности, и это применимо и здесь. Для аргументов, которые вы всё равно принимаете по ссылке (напомним, что `dyn Trait` не `Sized`, и что для его использования вам нужен толстый указатель), вы можете легко заменить ваш обобщённый аргумент на тот, что использует динамическую диспетчеризацию. Например, вместо `impl AsRef<str>` вы могли бы принять `&dyn AsRef<str>`.

Однако, прежде чем бросаться это делать, есть несколько вещей, которые вам стоит учесть. Во-первых, вы делаете этот выбор от имени ваших

пользователей, которые не смогут отказаться от динамической диспетчеризации. Если вы знаете, что код, к которому вы применяете динамическую диспетчеризацию, никогда не будет чувствителен к производительности, это, возможно, нормально. Но если придёт пользователь, который захочет использовать вашу библиотеку в своём высокопроизводительном приложении, динамическая диспетчеризация в функции, которая вызывается в горячем цикле, может стать препятствием. Во-вторых, на момент написания, использование динамической диспетчеризации будет работать только тогда, когда у вас есть простая граница трейта вроде `T: AsRef<str>` или `impl AsRef<str>`. Для более сложных границ Rust не знает, как сконструировать `vtable` динамической диспетчеризации, так что вы не можете принять, скажем, `&dyn Hash + Eq`. И наконец, помните, что при обобщениях вызывающий всегда может выбрать динамическую диспетчеризацию сам, передав объект-трейт (`trait object`). Обратное неверно: если вы принимаете объект-трейт, то именно его вызывающий и должен предоставить.

Может возникнуть соблазн начать ваши интерфейсы с конкретных типов, а затем со временем сделать их обобщёнными. Это может сработать, но имейте в виду, что такие изменения не обязательно обратно совместимы. Чтобы понять почему, представьте, что вы изменяете функцию с `fn foo(v: &Vec<usize>)` на `fn foo(v: impl AsRef<[usize]>)`. Хотя каждый `&Vec<usize>` реализует `AsRef<[usize]>`, вывод типов всё ещё может вызвать проблемы для пользователей. Рассмотрите, что произойдёт, если вызывающий вызовет `foo` с `foo(&iter.collect())`. В исходной версии компилятор мог определить, что ему следует собрать в `Vec`, но теперь он знает лишь, что ему нужно собрать в некоторый тип, реализующий `AsRef<[usize]>`. И таких типов может быть несколько, так что с этим изменением код вызывающего больше не скомпилируется!

Безопасность объекта (Object Safety)

Когда вы определяете новый трейт, является ли этот трейт безопасным для объектов (`object-safe`) (см. конец раздела «Компиляция и диспетчеризация» на странице 27 в главе 3) — это неписаная часть контракта трейта. Если трейт безопасен для объектов, пользователи могут трактовать различные типы, реализующие ваш трейт, как единый общий тип с помощью `dyn Trait`. Если нет, компилятор запретит `dyn Trait` для этого трейта. Вам следует предпочитать, чтобы ваши трейты были безопасны для объектов, даже если

это сопряжено с небольшой ценой для эргономики их использования (например, принятие `impl AsRef<str>` вместо `&str`), поскольку безопасность объектов открывает новые способы использования ваших трейтов. Если у вашего трейта обязательно должен быть обобщённый метод, рассмотрите, могут ли его обобщённые параметры быть на самом трейте, или могут ли его обобщённые аргументы тоже использовать динамическую диспетчеризацию, чтобы сохранить безопасность объектов трейта. В качестве альтернативы вы можете добавить границу трейта `where Self: Sized` к этому методу, что делает возможным вызов метода только с конкретным экземпляром трейта (а не через `dyn Trait`). Вы можете увидеть примеры этого паттерна в трейтах `Iterator` и `Read`, которые безопасны для объектов, но предоставляют некоторые дополнительные удобные методы на конкретных экземплярах.

Нет единственного ответа на вопрос о том, сколько жертв вам стоит быть готовым принести ради сохранения безопасности объектов. Моя рекомендация — учитывать, как ваш трейт будет использоваться, и имеет ли для пользователей смысл хотеть использовать его как объект-трейт. Если вы думаете, что пользователи, скорее всего, захотят использовать много разных типов, реализующих ваш трейт, вместе, вам стоит больше потрудиться над обеспечением безопасности объектов, чем если вы не думаете, что этот сценарий имеет большой смысл. Например, динамическая диспетчеризация не была бы полезна для трейта `FromIterator`, потому что его единственный метод не принимает `self`, так что вы и не смогли бы сконструировать объект-трейт в первую очередь. Аналогично, `std::io::Seek` сам по себе довольно бесполезен как объект-трейт, потому что единственное, что вы могли бы сделать с таким объектом-трейтом — это перемещаться (`seek`), не имея возможности читать или писать.

ОБЪЕКТЫ ТРЕЙТА DROP (DROP TRAIT OBJECTS) Вы можете подумать, что трейт `Drop` тоже бесполезен как объект-трейт, поскольку всё, что вы можете сделать с `Drop` как объектом-трейтом — это дропнуть его. Но оказывается, есть некоторые библиотеки, которые конкретно хотят иметь возможность дропать произвольные типы. Например, библиотека, которая предлагает отложенный дроп значений, как для конкурентной сборки мусора или просто отложенной очистки, заботится лишь о том, чтобы значения можно было дропнуть, и ни о чём больше. Что интересно, история `Drop` на этом не заканчивается; поскольку Rust должен иметь возможность дропать

и объекты-трейты, *каждая* `vtable` содержит метод дропа. Фактически, каждый `dyn Trait` также является `dyn Drop`.

Помните, что безопасность объектов — это часть вашего публичного интерфейса! Если вы модифицируете трейт обратно совместимым в остальном образом — например, добавляя метод с реализацией по умолчанию, но это делает трейт небезопасным для объектов — вам нужно повысить номер мажорной семантической версии.

Заемствованное против владеемого (Borrowed vs. Owned)

Почти для каждой функции, трейта и типа, которые вы определяете в Rust, вы должны решить, должны ли они владеть своими данными или просто держать на них ссылку. Какое бы решение вы ни приняли, оно будет иметь далеко идущие последствия для эргономики и производительности вашего интерфейса. К счастью, эти решения очень часто принимаются сами собой.

Если коду, который вы пишете, нужно владение данными — например, чтобы вызывать методы, принимающие `self`, или чтобы переместить данные в другой поток — он должен хранить владеемые данные. Когда ваш код должен владеть данными, ему, как правило, следует также заставить вызывающего предоставить владеемые данные, а не принимать значения по ссылке и клонировать их. Это оставляет контроль над выделением памяти за вызывающим и делает явной стоимость использования рассматриваемого интерфейса.

С другой стороны, если вашему коду не нужно владеть данными, ему следует оперировать ссылками. Одно распространённое исключение из этого правила — малые типы вроде `i32`, `bool` или `f64`, которые так же дёшево хранить и копировать напрямую, как и хранить через ссылки. Остерегайтесь, однако, предположения, что это справедливо для всех `Copy`-типов; `[u8; 8192]` является `Copy`, но хранить и копировать его повсюду было бы дорого.

Конечно, в реальном мире вещи часто менее однозначны. Иногда вы не знаете заранее, нужно ли вашему коду владеть данными или нет. Например, `String::from_utf8_lossy` нужно принять владение переданной ему последовательностью байтов только в том случае, если она содержит невалидные UTF-8 последовательности. В этом случае тип `Cow` — ваш друг: он позволяет вам оперировать ссылками, если данные это позволяют, и

позволяет вам произвести владеемое значение при необходимости.

В других случаях времена жизни (lifetime) ссылок настолько усложняют интерфейс, что им становится мучительно пользоваться. Если ваши пользователи борются с тем, чтобы заставить код скомпилироваться поверх вашего интерфейса, это знак того, что вам, возможно, стоит (даже без необходимости) принять владение определёнными фрагментами данных. Если вы делаете это, начните с данных, которые дёшево клонировать или которые не являются частью чего-либо чувствительного к производительности, прежде чем решите выделить в куче то, что может оказаться огромным куском байтов.

Способные к сбоям и блокирующие деструкторы (Fallible and Blocking Destructors)

Типы, ориентированные на ввод-вывод, часто нуждаются в выполнении очистки при дропе. Это может включать сброс записей на диск, закрытие файлов или корректное завершение соединений с удалёнными хостами. Естественное место для выполнения этой очистки — реализация `Drop` типа. К сожалению, как только значение дропнуто, у нас больше нет способа сообщить пользователю об ошибках, кроме как через панику. Похожая проблема возникает в асинхронном коде, где мы хотим завершить работу, когда есть ожидающая работа. К моменту вызова `drop` исполнитель (executor) может уже завершать работу, и у нас может не быть способа сделать больше. Мы могли бы попытаться запустить ещё один исполнитель, но это сопряжено со своим собственным набором проблем, таких как блокировка в асинхронном коде, как мы увидим в главе 9.

Нет идеального решения этих проблем, и что бы мы ни делали, некоторые приложения неизбежно вернутся к нашей реализации `Drop`. По этой причине нам нужно обеспечить очистку «по мере сил» через `Drop`. Если при очистке возникает ошибка, мы хотя бы попытались — мы проглатываем ошибку и продолжаем. Если исполнитель всё ещё доступен, мы можем породить будущее (future) для выполнения очистки, но если оно так и не запустится, мы сделали что смогли.

Однако нам следует предоставить лучшую альтернативу для пользователей, которые хотят не оставлять незавершённых концов. Мы можем сделать это,

предоставив явный деструктор. Обычно он принимает форму метода, который принимает владение `self` и раскрывает любые свои ошибки (используя `-> Result<_, _>`) или асинхронность (используя `async fn`), присущие этому уничтожению. Внимательный пользователь может затем использовать этот метод, чтобы корректно разобрать любые связанные ресурсы.

ПРИМЕЧАНИЕ Обязательно выделите явный деструктор в своей документации!

Как всегда, есть компромисс. В тот момент, когда вы добавляете явный деструктор, вы столкнётесь с двумя проблемами. Во-первых, поскольку ваш тип реализует `Drop`, вы больше не можете перемещать значения из любого из полей этого типа в деструкторе. Это потому, что `Drop::drop` всё ещё будет вызван после того, как отработает ваш явный деструктор, а он принимает `&mut self`, что требует, чтобы никакая часть `self` не была перемещена. Во-вторых, `drop` принимает `&mut self`, а не `self`, так что ваша реализация `Drop` не может просто вызвать ваш явный деструктор и проигнорировать его результат (потому что она не владеет `self`). Есть пара способов обойти эти проблемы, ни один из которых не идеален.

Первый — сделать ваш тип верхнего уровня `newtype`-обёрткой вокруг `Option`, который, в свою очередь, содержит некий внутренний тип, который хранит все поля типа. Затем вы можете использовать `Option::take` в обоих деструкторах и вызывать явный деструктор внутреннего типа только если внутренний тип ещё не был взят. Поскольку внутренний тип не реализует `Drop`, вы можете принять владение всеми полями там. Обратная сторона этого подхода в том, что все методы, которые вы хотите предоставить на типе верхнего уровня, теперь должны включать код, чтобы добраться через `Option` (который, как вы знаете, всегда `Some`, поскольку `drop` ещё не был вызван) до полей внутреннего типа.

Второй обходной путь — сделать каждое из ваших полей *взимаемым* (*takeable*). Вы можете «взять» `Option`, заменив его на `None` (что и делает `Option::take`), но вы можете делать это и со многими другими типами. Например, вы можете взять `Vec` или `HashMap`, просто заменив их на их дешёвые в конструировании значения по умолчанию — `std::mem::take` ваш друг здесь. Этот подход прекрасно работает, если у ваших типов есть осмысленные «пустые» значения, но становится утомительным, если вам нужно обернуть почти каждое поле в `Option`, а затем изменить каждый доступ к этим полям

соответствующим `unwrap`.

Третий вариант — хранить данные внутри типа `ManuallyDrop`, который разыменовывается во внутренний тип, так что нет нужды в `unwrap`. Вы также можете использовать `ManuallyDrop::take` в `drop`, чтобы принять владение во время уничтожения. Главный недостаток этого подхода в том, что `ManuallyDrop::take` небезопасен (`unsafe`). Нет встроенных механизмов безопасности, гарантирующих, что вы не попытаетесь использовать значение внутри `ManuallyDrop` после того, как вызвали `take`, или что вы не вызовете `take` несколько раз. Если вы это сделаете, ваша программа будет молча проявлять неопределённое поведение (`undefined behavior`), и случатся плохие вещи.

В конечном счёте, вам следует выбрать тот из этих подходов, который лучше всего подходит вашему приложению. Я бы склонился к второму варианту, переключаясь на другие, если обнаружу себя в море `Option`. Решение с `ManuallyDrop` превосходно, если код достаточно прост, чтобы вы могли легко проверить безопасность вашего кода, и вы уверены в своей способности это сделать.

Очевидный (Obvious)

Хотя некоторые пользователи могут быть знакомы с аспектами реализации, лежащей в основе вашего интерфейса, они вряд ли поймут все его правила и ограничения. Они не будут знать, что вызывать `foo` после вызова `bar` никогда нельзя, или что вызывать небезопасный метод `baz` безопасно только когда луна находится под углом 47 градусов и никто не чихал в последние 18 секунд. Только если интерфейс ясно даёт понять, что происходит нечто странное, они потянутся за документацией или внимательно прочитают сигнатуры типов. Поэтому критически важно, чтобы вы делали ваш интерфейс максимально простым для понимания пользователями и максимально сложным для некорректного использования. Два основных приёма в вашем распоряжении для этого — это ваша документация и система типов, так что давайте рассмотрим каждый из них по очереди.

ПРИМЕЧАНИЕ Вы также можете воспользоваться именованием, чтобы намекнуть пользователю, что в интерфейсе есть нечто большее, чем кажется на первый взгляд. Если пользователь видит метод под названием `dangerous`, велика вероятность, что он прочтёт его

Документация (Documentation)

Первый шаг к тому, чтобы сделать ваши интерфейсы прозрачными — это написать хорошую документацию. Я мог бы написать целую книгу, посвящённую тому, как писать документацию, но давайте сосредоточимся здесь на специфичных для Rust советах.

Во-первых, чётко документируйте любые случаи, когда ваш код может сделать нечто неожиданное, или когда он полагается на то, что пользователь сделает нечто сверх того, что диктует сигнатура типа. Паники — хороший пример обоих этих обстоятельств: если ваш код может паниковать, задокументируйте этот факт вместе с обстоятельствами, при которых он может паниковать. Аналогично, если ваш код может вернуть ошибку, задокументируйте случаи, в которых это происходит. Для небезопасных функций задокументируйте, что вызывающий должен гарантировать, чтобы вызов был безопасным.

Во-вторых, включайте сквозные (end-to-end) примеры использования вашего кода на уровне крейта и модуля. Они важнее, чем примеры для конкретных типов или методов, поскольку они дают пользователю представление о том, как всё сочетается вместе. С приличным высокоуровневым пониманием структуры интерфейса разработчик может вскоре осознать, что делают конкретные методы и типы и где их следует использовать. Сквозные примеры также дают пользователю отправную точку для настройки своего использования, и они могут, и часто будут, скопировать пример, а затем модифицировать его под свои нужды. Этот вид «обучения на практике» обычно работает лучше, чем заставлять их собирать всё воедино из компонентов.

ПРИМЕЧАНИЕ Очень специфичные для метода примеры, которые показывают, что да, метод `len` действительно возвращает длину, вряд ли скажут пользователю что-то новое о вашем коде.

В-третьих, организуйте вашу документацию. Когда все ваши типы, трейты и функции находятся в едином модуле верхнего уровня, пользователю трудно понять, с чего начать. Воспользуйтесь модулями, чтобы сгруппировать семантически связанные элементы. Затем используйте внутридокументационные ссылки (intra-documentation links), чтобы связать

элементы между собой. Если в документации на тип A говорится о трейте B, то она должна ссылаться прямо на этот трейт. Если вы облегчаете пользователю исследование вашего интерфейса, он с меньшей вероятностью пропустит важные связи или зависимости. Также рассмотрите возможность пометить части вашего интерфейса, которые не предназначены для публичности, но нужны по причинам обратной совместимости, с помощью `#[doc(hidden)]`, чтобы они не загромождали вашу документацию.

И наконец, обогащайте вашу документацию, где это возможно. Ссылайтесь на внешние ресурсы, которые объясняют концепции, структуры данных, алгоритмы или другие аспекты вашего интерфейса, которые могут иметь хорошие объяснения в другом месте. RFC, посты в блогах и официальные документы (whitepapers) отлично подходят для этого, если они релевантны. Используйте `#[doc(cfg(..))]`, чтобы выделить элементы, доступные только при определённых конфигурациях, чтобы пользователь быстро понял, почему некоторый метод, перечисленный в документации, недоступен. Используйте `#[doc(alias = "...")]`, чтобы сделать типы и методы обнаруживаемыми под другими именами, по которым пользователи могут их искать. В документации верхнего уровня направляйте пользователя к часто используемым модулям, фичам, типам, трейтам и методам.

Руководство со стороны системы типов (Type System Guidance)

Система типов — это превосходный инструмент для того, чтобы гарантировать, что ваши интерфейсы очевидны, самодокументируемы и устойчивы к некорректному использованию. В вашем распоряжении есть несколько приёмов, которые могут сделать ваши интерфейсы очень трудными для некорректного использования и тем самым повысить вероятность того, что они будут использоваться правильно.

Первый из них — это *семантическая типизация (semantic typing)*, при которой вы добавляете типы, чтобы представить *смысл* значения, а не только его примитивный тип. Классический пример здесь — для булевых значений: если ваша функция принимает три аргумента `bool`, есть вероятность, что какой-нибудь пользователь перепутает порядок значений и осознает это только после того, как что-то пошло ужасно не так. Если же, с другой стороны, она принимает три аргумента отдельных перечислений из двух вариантов, пользователь не сможет перепутать порядок без того, чтобы компилятор не

накричал на него: если он попытается передать `DryRun::Yes` в аргумент `overwrite`, это просто не сработает, как и передача `Overwrite::No` в аргумент `dry_run`. Вы можете применять семантическую типизацию и за пределами булевых значений. Например, `newtype` вокруг числового типа может предоставить единицу измерения для содержащегося значения, или может ограничить аргументы сырых указателей только теми, что были возвращены другим методом.

Тесно связанный приём — использовать типы нулевого размера (zero-sized types), чтобы указать, что определённый факт верен относительно экземпляра типа. Рассмотрим, например, тип под названием `Rocket`, который представляет состояние настоящей ракеты. Некоторые операции (методы) на `Rocket` должны быть доступны независимо от того, в каком состоянии находится ракета, но некоторые имеют смысл только в определённых ситуациях. Например, невозможно запустить ракету, если она уже запущена. Аналогично, вероятно, не должно быть возможным отделить топливный бак, если ракета ещё не запущена. Мы могли бы смоделировать это как варианты перечисления, но тогда все методы были бы доступны на каждом этапе, и нам пришлось бы вводить возможные паники.

Вместо этого, как показано в листинге 4-2, мы можем ввести обобщённый параметр на `Rocket`, `Stage`, и использовать его, чтобы ограничить, какие методы доступны и когда.

```
❶ struct Grounded;
struct Launched;
// и так далее
struct Rocket<Stage = Grounded> {
❷ stage: std::marker::PhantomData<Stage>,
}

❸ impl Default for Rocket<Grounded> {}
impl Rocket<Grounded> {
    pub fn launch(self) -> Rocket<Launched> { }
}
❹ impl Rocket<Launched> {
    pub fn accelerate(&mut self) { }
    pub fn decelerate(&mut self) { }
}

❺ impl<Stage> Rocket<Stage> {
    pub fn color(&self) -> Color { }
    pub fn weight(&self) -> Kilograms { }
}
```

Листинг 4-2: Использование типов-маркеров для ограничения реализаций

Мы вводим единичные типы (unit types), чтобы представить каждый этап ракеты ❶. Нам на самом деле не нужно хранить этап — нам нужна лишь предоставляемая им метайнформация — так что мы храним его за `PhantomData` ❷, чтобы гарантировать, что он будет устранён во время компиляции. Затем мы пишем блоки реализации для `Rocket` только тогда, когда он содержит определённый типовой параметр. Вы можете сконструировать ракету только на земле (пока что), и вы можете запустить её только с земли ❸. Только когда ракета запущена, вы можете управлять её скоростью ❹. Есть некоторые вещи, которые вы всегда можете делать с ракетой, в каком бы состоянии она ни находилась, и их мы помещаем в обобщённый блок реализации ❺. Вы заметите, что при таком спроектированном интерфейсе для пользователя просто невозможно вызвать метод в неправильное время — мы закодировали правила использования в самих типах и сделали недопустимые состояния *непредставимыми* (*unrepresentable*).

Это понятие распространяется и на многие другие предметные области; если ваша функция игнорирует аргумент-указатель, если только заданный булев аргумент не истинен, лучше вместо этого объединить два аргумента. С типом-перечислением, у которого один вариант для `false` (и без указателя) и один вариант для `true`, который содержит указатель, ни вызывающий, ни реализующий не могут неправильно понять отношение между ними. Это мощная идея, использование которой я настоятельно рекомендую.

Ещё один маленький, но полезный инструмент для того, чтобы сделать интерфейсы очевидными — это аннотация `#[must_use]`. Добавьте её к любому типу, трейту или функции, и компилятор выдаст предупреждение, если код пользователя получает элемент этого типа или трейта, или вызывает эту функцию, и не обрабатывает его явно. Вы, возможно, уже видели это в контексте `Result`: если функция возвращает `Result`, а вы не присваиваете его возвращаемое значение куда-либо, вы получаете предупреждение компилятора. Будьте осторожны, чтобы не злоупотреблять этой аннотацией — добавляйте её только если пользователь с очень большой вероятностью совершит ошибку, не используя возвращаемое значение.

Ограниченный (Constrained)

Со временем какой-нибудь пользователь начнёт зависеть от каждого свойства вашего интерфейса, будь то баг или фича. Это особенно верно для публично доступных библиотек, где у вас нет контроля над вашими пользователями. В результате вам следует тщательно подумать, прежде чем вносить видимые пользователю изменения. Добавляете ли вы новый тип, поле, метод или реализацию трейта, или изменяете существующие — вы хотите убедиться, что изменение не сломает код существующих пользователей и что вы планируете сохранять это изменение какое-то время. Частые обратно несовместимые изменения (увеличения мажорной версии в семантическом версионировании) наверняка навлекут на вас гнев ваших пользователей.

Многие обратно несовместимые изменения очевидны, как переименование публичного типа или удаление публичного метода, но некоторые более тонкие и глубоко завязаны на то, как работает Rust. Здесь мы рассмотрим некоторые из более заковыристых тонких изменений и то, как к ним подготовиться. Вы увидите, что вам нужно балансировать некоторые из них с тем, насколько гибким вы хотите видеть свой интерфейс — иногда чем-то приходится поступиться.

Модификации типов (Type Modifications)

Удаление или переименование публичного типа почти наверняка сломает чей-то код. Чтобы противостоять этому, вы захотите воспользоваться модификаторами видимости Rust, такими как `pub(crate)` и `pub(in path)`, везде, где это возможно. Чем меньше у вас публичных типов, тем больше у вас свободы изменять вещи позднее, не ломая существующий код.

Однако код пользователя может зависеть от ваших типов не только по имени. Рассмотрите публичный тип в листинге 4-3 и данное использование этого кода.

```
// в вашем интерфейсе
pub struct Unit;
// в коде пользователя
let u = lib::Unit;
```

Листинг 4-3: Невинно выглядящий публичный тип

Теперь рассмотрим, что произойдёт, если вы добавите приватное поле к `Unit`. Хотя поле, которое вы добавляете, является приватным, изменение всё равно сломает код пользователя, потому что конструктор, на который он полагался, исчез. Аналогично рассмотрите код и использование в листинге 4-4.

```
// в вашем интерфейсе
pub struct Unit { pub field: bool };
// в коде пользователя
fn is_true(u: lib::Unit) -> bool {
    matches!(u, Unit { field: true })
}
```

Листинг 4-4: Код пользователя, обращающийся к единственному публичному полю

Здесь тоже добавление приватного поля к `Unit` сломает код пользователя, на этот раз потому что логика проверки исчерпывающего сопоставления с образцом (pattern matching) Rust способна видеть части интерфейса, которые пользователь видеть не может. Она распознаёт, что есть больше полей, даже несмотря на то что код пользователя не может к ним обратиться, и отвергает образец пользователя как неполный. Похожая проблема возникает, если мы превращаем кортежную структуру (tuple struct) в обычную структуру с именованными полями: даже если сами поля точно те же, любые старые образцы больше не будут валидны для нового определения типа.

Rust предоставляет атрибут `#[non_exhaustive]`, чтобы помочь смягчить эти проблемы. Вы можете добавить его к любому определению типа, и компилятор запретит использование неявных конструкторов (вроде `lib::Unit { field1: true }`) и неисчерпывающих сопоставлений с образцом (то есть образцов без замыкающего `, ..`) на этом типе. Это отличный атрибут для добавления, если вы подозреваете, что, вероятно, будете модифицировать определённый тип в будущем. Он, однако, ограничивает код пользователя, например, лишая пользователей возможности полагаться на исчерпывающие сопоставления с образцом, так что избегайте его добавления, если думаете, что данный тип, скорее всего, останется стабильным.

Реализации трейтов (Trait Implementations)

Как вы помните из главы 3, правила когерентности Rust запрещают множественные реализации данного трейта для данного типа. Поскольку мы

не знаем, какие реализации мог добавить нижестоящий (downstream) код, добавление бланкетной реализации существующего трейта обычно является ломающим изменением. То же верно для реализации чужого трейта для существующего типа или существующего трейта для чужого типа — в обоих случаях владелец чужого трейта или типа может одновременно добавить конфликтующую реализацию, так что это должно быть ломающим изменением.

Удаление реализации трейта является ломающим изменением, но реализация трейтов для *нового* типа никогда не является проблемой, поскольку ни один крейт не может иметь реализаций, конфликтующих с этим типом.

Возможно, контринтуитивно, но вам также стоит быть осторожным с реализацией *любого* трейта для существующего типа. Чтобы понять почему, рассмотрите код в листинге 4-5.

```
// crate1 1.0
pub struct Unit;
pub trait Fool { fn foo(&self) }
// обратите внимание, что Fool не реализован для Unit

// crate2; зависит от crate1 1.0
use crate1::{Unit, Fool};
trait Foo2 { fn foo(&self) }
impl Foo2 for Unit { .. }
fn main() {
    Unit.foo();
}
```

Листинг 4-5: Реализация трейта для существующего типа может вызвать проблемы.

Если вы добавите `impl Fool for Unit` в `crate1`, не пометив это как ломающее изменение, нижестоящий код внезапно перестанет компилироваться, поскольку вызов `foo` теперь неоднозначен. Это может применяться даже к реализациям *новых* публичных трейтов, если нижестоящий крейт использует импорты по шаблону (`use crate1::*`). Вы особенно захотите держать это в уме, если предоставляете модуль `prelude`, который инструктируете пользователей использовать с импортами по шаблону.

Большинство изменений существующих трейтов также являются ломающими изменениями, как изменение сигнатуры метода или добавление нового

метода. Изменение сигнатуры метода ломает все реализации и, вероятно, многие использования трейта, тогда как добавление нового метода «всего лишь» ломает все реализации. Добавление нового метода с реализацией по умолчанию, однако, нормально, поскольку существующие реализации продолжают применяться.

Я говорю «обычно» и «большинство» здесь, потому что, будучи авторами интерфейса, у нас есть доступный инструмент, который позволяет нам обойти некоторые из этих правил: *запечатанные трейты (sealed traits)*. Запечатанный трейт — это такой, который может быть использован, но не реализован, другими крейтами. Это немедленно делает ряд ломающих изменений неломаящими. Например, вы можете добавить новый метод к запечатанному трейту, поскольку знаете, что нет реализаций вне текущего крейта, которые стоило бы учитывать. Аналогично вы можете реализовать запечатанный трейт для новых чужих типов, поскольку знаете, что чужой крейт, определивший этот тип, не мог добавить конфликтующую реализацию.

Запечатанные трейты чаще всего используются для *выводимых (derived)* трейтов — трейтов, которые предоставляют бланкетные реализации для типов, реализующих определённые другие трейты. Вам следует запечатывать трейт только если для чужого крейта не имеет смысла реализовывать ваш трейт; это серьёзно ограничивает полезность трейта, поскольку нижестоящие крейты больше не смогут реализовывать его для своих собственных типов. Вы также можете использовать запечатанные трейты, чтобы ограничить, какие типы можно использовать как типовые аргументы, например ограничивая тип Stage в примере с Rocket из листинга 4-2 только типами Grounded и Launched.

Листинг 4-6 показывает, как запечатать трейт и как затем всё же добавить реализации для него.

```
pub trait CanUseCannotImplement: sealed::Sealed ❶ { .. }
mod sealed {
  pub trait Sealed {}
  ❷ impl<T> Sealed for T where T: TraitBounds {}
}
impl<T> CanUseCannotImplement for T where T: TraitBounds {}
```

Листинг 4-6: Как запечатать трейт и добавить для него реализации

Трюк состоит в том, чтобы добавить приватный, пустой трейт в качестве

супертрейта (supertrait) того трейта, который вы хотите запечатать ❶. Поскольку супертрейт находится в приватном модуле, другие крейты не могут до него добраться и потому не могут его реализовать. Запечатанный трейт требует, чтобы нижележащий тип реализовывал `Sealed`, так что только те типы, которым мы явно разрешаем ❷, способны в конечном счёте реализовать трейт.

ПРИМЕЧАНИЕ Если вы запечатываете трейт таким образом, обязательно задокументируйте этот факт, чтобы пользователи не расстраивались, пытаясь реализовать трейт сами!

Скрытые контракты (Hidden Contracts)

Иногда изменения, которые вы вносите в одну часть вашего кода, тонкими способами влияют на контракт в другом месте вашего интерфейса. Два основных способа, которыми это происходит — через реэкспорты (re-exports) и авто-трейты.

Реэкспорты (Re-Exports)

Если какая-либо часть вашего интерфейса раскрывает чужие типы, то любое изменение одного из этих чужих типов является *также* изменением вашего интерфейса. Например, рассмотрите, что произойдёт, если вы перейдёте на новую мажорную версию зависимости и раскроете тип из этой зависимости как, скажем, тип итератора в вашем интерфейсе. Пользователь, который зависит от вашего интерфейса, может также зависеть напрямую от этой зависимости и ожидать, что тип, который предоставляет ваш интерфейс — тот же самый, что и тип с тем же именем в этой зависимости. Но если вы измените мажорную версию вашей зависимости, это больше не будет верным, даже несмотря на то что *имя* типа осталось тем же. Листинг 4-7 показывает пример этого.

```
// ваш крейт: bestiter
pub fn iter<T>() -> itercrate::Empty<T> { .. }
// их крейт
struct EmptyIterator { it: itercrate::Empty<()> }
EmptyIterator { it: bestiter::iter() }
```

Листинг 4-7: Реэкспорты делают чужие крейты частью контракта вашего интерфейса.

Если ваш крейт перейдёт с `itercrate 1.0` на `itercrate 2.0`, но в остальном не изменится, код в этом листинге больше не скомпилируется. Хотя никакие типы не изменились, компилятор (правильно) считает, что `itercrate1.0::Empty` и `itercrate2.0::Empty` — это *разные* типы. Поэтому вы не можете присвоить последний первому, что делает это ломающим изменением в вашем интерфейсе.

Чтобы смягчить такого рода проблемы, часто лучше обернуть чужие типы, используя паттерн `newtype`, а затем раскрывать только те части чужого типа, которые вы считаете полезными. Во многих случаях вы можете вообще избежать `newtype`-обёртки, используя `impl Trait`, чтобы предоставить вызывающему лишь самый минимальный контракт. Обещая меньше, вы делаете меньше изменений ломающими.

ТРЮК SEMVER (THE SEMVER TRICK) Пример с `itercrate` мог вас покори́ть. Если тип `Empty` не изменился, то почему компилятор не позволяет ничему, что его использует, продолжать работать, независимо от того, использует ли код версию 1.0 или 2.0? Ответ... сложный. Он сводится к тому факту, что компилятор Rust не предполагает, что только потому, что два типа имеют одинаковые поля, они одинаковы. Возьмём простой пример этого: представьте, что `itercrate 2.0` добавил `#[derive(Copy)]` для `Empty`. Теперь у типа внезапно разная семантика перемещения в зависимости от того, используете ли вы 1.0 или 2.0! И код, написанный с расчётом на одну, не будет работать с другой.

Эта проблема имеет тенденцию возникать в крупных, широко используемых библиотеках, где со временем ломающим изменениям, вероятно, придётся происходить *где-нибудь* в крейте. К сожалению, семантическое версионирование происходит на уровне крейта, а не на уровне типа, так что ломающее изменение где угодно является ломающим изменением везде.

Но не всё потеряно. Несколько лет назад Дэвид Толнай (David Tolnay) (автор `serde`, среди огромного числа других вкладов в Rust) придумал изящный трюк для обработки именно такого рода ситуации. Он назвал его «трюком `semver`». Идея проста: если некий тип `T` остаётся неизменным через ломающее изменение (скажем, с 1.0 до 2.0), то после выпуска 2.0 вы можете выпустить новую *минорную* версию 1.0,

которая зависит от 2.0 и заменяет `T` реэкспортом `T` из 2.0.

Делая это, вы гарантируете, что на самом деле существует только один тип `T` через обе мажорные версии. Это, в свою очередь, означает, что любой крейт, зависящий от 1.0, сможет использовать `T` из 2.0, и наоборот. И поскольку это происходит только для типов, для которых вы *явно* соглашаетесь на этот трюк, изменения, которые на самом деле были ломающими, такими и останутся.

Авто-трейты (Auto-Traits)

В Rust есть горстка трейтов, которые автоматически реализуются для каждого типа в зависимости от того, что этот тип содержит. Наиболее релевантные из них для этого обсуждения — `Send` и `Sync`, хотя трейты `Unpin`, `Sized` и `UnwindSafe` имеют похожие проблемы. По своей природе эти трейты добавляют скрытое обещание, даваемое почти каждым типом в вашем интерфейсе. Эти трейты даже распространяются через типы со стёртыми типами (type-erased), вроде `impl Trait`.

Реализации этих трейтов (как правило) автоматически добавляются компилятором, но это также означает, что они *не* добавляются автоматически, если они больше не применимы. Так что если у вас есть публичный тип `A`, который содержит приватный тип `B`, и вы изменяете `B` так, что он больше не `Send`, то `A` теперь *тоже* не `Send`. Это ломающее изменение!

За такими изменениями может быть трудно уследить, и они часто не обнаруживаются, пока пользователь вашего интерфейса не пожалуется, что его код больше не работает. Чтобы отлавливать эти случаи, хорошая практика — включить в ваш набор тестов несколько простых тестов, которые проверяют, что все ваши типы реализуют эти трейты так, как вы ожидаете. Листинг 4-8 даёт пример того, как мог бы выглядеть такой тест.

```
fn is_normal<T: Sized + Send + Sync + Unpin>() {}
#[test]
fn normal_types() {
    is_normal::<MyType>();
}
```

Листинг 4-8: Тестирование того, что тип реализует набор трейтов

Обратите внимание, что этот тест не выполняет никакого кода, а просто

проверяет, что код компилируется. Если `MyType` больше не реализует `Sync`, тестовый код не скомпилируется, и вы узнаете, что изменение, которое вы только что сделали, сломало реализацию авто-трейта.

СКРЫТИЕ ЭЛЕМЕНТОВ ИЗ ДОКУМЕНТАЦИИ (HIDING ITEMS FROM DOCUMENTATION)

Атрибут `#[doc(hidden)]` позволяет вам скрыть публичный элемент из вашей документации, не делая его недоступным для кода, который случайно знает, что он есть. Это часто используется, чтобы раскрыть методы и типы, которые нужны макросам, но не коду пользователя. То, как такие скрытые элементы взаимодействуют с контрактом вашего интерфейса — предмет некоторых споров. В общем, элементы, помеченные как `#[doc(hidden)]`, считаются частью вашего контракта лишь постольку, поскольку касается их публичных эффектов; например, если код пользователя может в итоге содержать скрытый тип, то является ли этот тип `Send` или нет — часть контракта, тогда как его имя — нет. Скрытые собственные методы и скрытые методы трейтов на запечатанных трейтах, как правило, не являются частью контракта вашего интерфейса, хотя вам следует чётко указать это в документации к этим методам. И да, скрытые элементы всё равно следует документировать!

Итоги (Summary)

В этой главе мы исследовали множество граней проектирования интерфейса Rust, будь то предназначен он для внешнего использования или просто как граница абстракции между различными модулями внутри вашего крейта. Мы охватили много конкретных подводных камней и трюков, но в конечном счёте именно высокоуровневые принципы должны направлять ваше мышление: ваши интерфейсы должны быть без сюрпризов, гибкими, очевидными и ограниченными. В следующей главе мы углубимся в то, как представлять и обрабатывать ошибки в коде Rust.